Nurse Handoff and Patient Status Management System for a Hospital

Project Proposal for UCS503P
Submitted to: Anushka Choudhary
Thapar Institute of Engineering and Technology

Arnav Sharma Dhruv Dwivedi Pratham Nandwani Jalaj Poonia

August 25, 2026

1 Higher-order goal

. . . secondary framing

Nurse shift handoffs are where a lot of patient information quietly falls through the cracks. This project tries to fix that by giving nurses a structured way to pass on patient status instead of relying on memory, a scribbled note, or a rushed verbal update at the end of a 12-hour shift. Put simply: fewer things get missed when a patient is handed from one nurse to the next.

2 Time-to-value

. . . primary heuristic

- Instead of designing the whole system upfront, we want a working version of the core loop (a nurse sends a handoff, another nurse accepts or flags it) as early as possible, so we can actually see if it holds up in practice.
- We're going with weekly iterations and setting up CI early so that every change is testable and shippable, rather than saving everything for one big release.
- What counts as "done" for the first iteration is deliberately small — we'd rather ship something narrow that works than something broad that half-works.

3 Problem Statement

Talk to almost any nurse and they'll tell you handoffs are stressful and error-prone. A few recurring issues:

- **Nothing is written down consistently.** Handoffs happen verbally, on sticky notes, or from memory, so there's no single place that holds what actually needs to be passed on.
- **Quality depends on the nurse.** A senior nurse's handoff might be thorough; a rushed one at the end of a long shift might skip the one detail that mattered.
- **No record of who said what.** If something goes wrong later, there's no way to check what was actually communicated, by whom, or whether the next nurse even acknowledged it.
- **No way to push back.** If a handoff is unclear, incomplete, or meant for someone else entirely, there's currently no clean way to flag that and get it sorted.

- **Doctors are stuck piecing things together.** A doctor rounding on a patient often has to reconstruct the last day or two from multiple nurses across multiple shifts.

Why this matters: handoffs are a known weak point in hospital care — a lot of preventable mistakes trace back to information that simply didn't make it from one shift to the next. Even a fairly simple fix here (structure, a record, and a way to flag problems) can meaningfully cut down on things slipping through.

4 Proposed Solution *. . . Software Proposition*

4.1 Overview

We're building a **web-based** nurse workflow and patient-status system with five main pieces:

- **A nurse's own patient view.** Every nurse can see the patients currently under her care, and structurally view and log against them — building up notes and observations over the course of the shift, not just scrambling to write something at handoff time.
- **Alerting the doctor.** From that same patient view, an **Alert** button lets her flag the assigned doctor directly if something needs attention, with an optional text box to add a short message along with the alert.
- **Voice-first note capture.** She can talk through a note instead of typing it, and the system transcribes and roughly structures it. Typing/editing is always available too, as one input option among others.
- **Handoff.** At shift end, she picks which patients to hand off and sends each one's structured note — Details and Major Decisions — to the incoming nurse, who reviews it alongside the full *See Log* and then Accepts, Declines, or Reports an Issue.
- **Doctor screen.** Shows every patient under that doctor, with each patient's structured log sitting right there, so the doctor isn't hunting for context.
- **Admin screen.** Add or remove nurses, and assign which nurse and which doctor a patient belongs to.

4.2 Core workflow

1. A nurse opens her own patient view and, over the course of the shift, builds up a structured log for each patient under her — by voice or by typing — organized into Details and Major Decisions as she goes.
2. If something comes up that needs the doctor's attention right away, she hits **Alert** on that patient, optionally adds a short message, and it's sent straight to the assigned doctor.
3. When it's time to hand off (end of shift, or whenever), she picks the relevant patients and gives the structured note a quick review/edit.
4. She sends it to the incoming nurse.
5. The incoming nurse opens each patient and actually reads through Details, Major Decisions, and the full Log before doing anything.
6. She then either **Accepts** (takes over the patient), **Declines** (this shouldn't have come to her, or it's missing too much), or **Reports an Issue** (something's wrong or unclear, but it still needs to be sorted out rather than just rejected).

7. Whenever the doctor checks in on a patient, the structured log is already sitting there for them.

8. The admin, meanwhile, handles the boring-but-necessary part: adding/removing nurse accounts and keeping patient-nurse-doctor assignments correct.

4.3 A few constraints worth calling out

- It needs to work fine on whatever hardware the ward actually has — shared tablets, older workstations — so we're keeping the frontend responsive and light.
- Voice is a convenience, not a dependency. A nurse should never be stuck because the transcription got something wrong — she can always type or fix it before sending. Given this is patient care, we're not willing to auto-send anything without her actually reviewing it first.
- The system isn't trying to be clinically smart — it's not making decisions or diagnosing anything. It just carries forward what the nurse actually said, in a structured and accountable way.
- We're aiming for something that can be hosted normally (managed DB, standard web hosting) with proper access control, given this is patient data.

5 Solution Approach *... Engineering focus*

This is an engineering project, not a research one, so the effort goes into getting the workflow, accountability, and correctness right rather than building anything clever on the speech/NLP side.

5.1 Voice capture and structuring *... non-research*

- We'll use an off-the-shelf speech-to-text service for transcription — there's no reason to build our own here.
- Sorting the text into Details vs. Major Decisions can start as something the nurse controls directly (she picks the section), with basic keyword-based suggestions added later if there's time.
- Nothing gets sent without the nurse seeing and approving the final text first — that's a hard rule, not a nice-to-have.

5.2 Web architecture *... web-based solution*

Standard 3-tier setup:

- **Frontend:** separate views for nurse (send/receive), doctor (patient logs), and admin (accounts/assignments), all role-gated.
- **Backend API:** auth, role checks, handoff creation, the Accept/Decline/Report Issue transitions, and assignment logic.
- **Database:** users and roles, patients, handoffs and their structured notes, the audit log, and assignment records.

5.3 CI/CD

Nothing fancy — just the basics that let us move fast without breaking things:

- Tests run automatically on every push.
- Every merge produces a deployable build.
- Staging deploys are a single pipeline step, not a manual chore.

6 Evaluation Criterion *... measurable and attributable*

6.1 Primary metric

Time-to-acknowledgement (TTA): how long between a handoff being sent and the recipient actually responding (Accept, Decline, or Report Issue).

- Rough target: median TTA under 15 minutes during the pilot — open to adjusting once we see real numbers.
- Pulled straight from timestamps in the database, so it's easy to verify.

6.2 Secondary metrics

- **Clean-accept rate:** how many handoffs go through without a Report Issue afterward.
- **Voice usage and correction rate:** how often nurses actually use voice vs. typing, and how much they end up correcting the transcription.
- **Doctor feedback:** a quick 1–5 rating on whether the consolidated logs are actually useful before rounds.
- **Uptime:** the system needs to be up when it matters most — during shift changes — so we'll track availability specifically in that window.

6.3 How we'll actually test this *... high-level*

- Get 10–15 nurses (or run a simulated ward if that's not possible), a couple of doctor roles, and one admin involved in a pilot.
- Compare against how long/complete handoffs are today, based on quick interviews or just observing the current process, and see if the numbers actually move.

7 Scalability *... with foresight*

1. **In theory:** it should hold up as the number of patients, nurses, and handoffs grows — so frontend and backend stay decoupled, queries on logs/handoffs are indexed and paginated, and the API stays stateless.
2. **In practice:** it should actually be deployable without heroics — managed database with encryption, containerized deployment, CI/CD into staging.

8 Engine availability heuristic

We're not short on tools for any of this:

- A speech-to-text API for the voice capture.
- A standard web framework for the REST API and the role-based screens.
- A managed relational or document database.
- Auth/role handling for nurse, doctor, and admin.
- Object storage for attachments, if we get to that (local storage is fine for the pilot).
- A test framework for the backend.
- A CI runner and a staging target to deploy to.

None of this needs to be built from scratch — the actual work is in getting the workflow and the accountability model right.

9 Project scope and deliverables *. . . iteration-friendly*

9.1 First iteration *. . . initial deliverable*

- Sending a handoff: pick patients, write/edit Details and Major Decisions, send.
- Receiving a handoff: Details / Major Decisions / See Log, plus Accept, Decline, Report Issue.
- Core data model: users, patients, handoffs, assignments.
- Timestamps everywhere they're needed for TTA.
- CI: build, backend tests, linting.
- One-step deploy to staging.

9.2 Later

- Voice capture, with the mandatory review step before sending.
- Alert button on the nurse's patient view, notifying the assigned doctor with an optional message.
- Doctor screen with the consolidated per-patient logs.
- Admin screen: nurse management and patient/nurse/doctor assignment.
- In-app notifications for new, declined, or disputed handoffs.
- Better search/filtering and indexing as data grows.

10 Risks and mitigations

- **Patient data is sensitive.** Role-based access, encryption in transit and at rest, and we only collect what's actually needed — plus an audit trail of who touched what.
- **Voice transcription can get things wrong, and this is a hospital.** No auto-sending, ever. The nurse always reviews before it goes out.
- **Nurses are busy and won't use something clunky.** Keep the send/receive flow as short as possible, and treat voice as optional, not required.
- **"Did it actually work" can get fuzzy if we're not careful.** Nail down exactly what Accept/Decline/Report Issue mean and when each timestamp fires, before we start building.
- **Things break during a pilot.** Get staging running early, keep CI/CD reliable, and specifically watch uptime during shift-change hours since that's when it matters most.

11 Summary

This is a web platform for structuring and tracking nurse shift handoffs — voice-first note capture, a three-part review for the receiving nurse (Details, Major Decisions, Log), explicit Accept/Decline/Report Issue accountability, a consolidated view for doctors, and admin-managed assignments — aimed at cutting down how much gets lost between shifts. It's built around fast iterations, CI/CD for safe frequent releases, and a clear metric (time-to-acknowledgement) to actually tell if it's working, rather than betting everything on a big upfront design.